
Anomaly Detection using Negative Selection Algorithms

Vincent Degenhart
Radboud University
vincent.degenhart@ru.nl

Nuno Moreira Silva
Radboud University
nuno.moreirasilva@ru.nl

Teo Pazera
Radboud University
teo.pazera@ru.nl

1 Introduction

This report applies the negative selection algorithm to anomaly detection in sequence data. The main focus is the intrusion detection task on Unix system calls (variable-length sequences). In addition, the language classification toy example is included in Appendix A as required, where ROC curves and AUC are used to analyze discrimination performance.

2 Methods

Dataset and Evaluation

We use the dataset by Stephanie Forrest’s Lab [1], which consists of sequences of system calls. A preliminary analysis of the dataset reveals a significant variance in sequence lengths, as illustrated in Figure 1. To quantitatively assess the algorithm’s performance, we use the scikit-learn library [2].

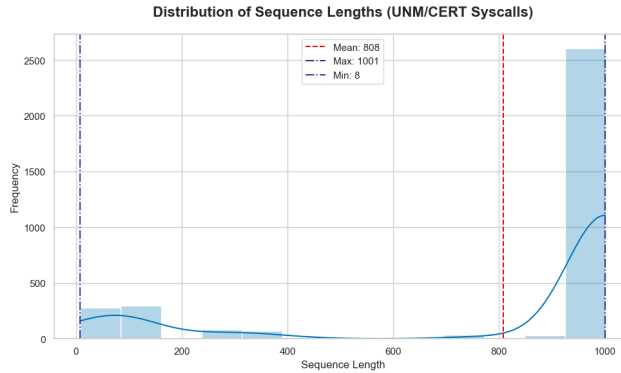


Figure 1: Distribution of system call sequence lengths for the UNM/CERT dataset. The sequences have a wide range ($L_{min} = 8$, $L_{max} = 1001$) with a significant concentration of traces at the maximum length. This high variance in L necessitates the use of a length-normalized anomaly score, $A(S)$, to ensure fair comparison between short and long execution traces.

Parameter Search

We perform a naive parameter search, following the guidelines acquired from the previous task. Namely, that we want to choose n and r in such a way that (i) $n > r$, (ii) n is not too big in comparison to r , and (iii) $n \geq 8$ (Appendix B).

Sliding Window Approach

To account for varying sequence lengths, we implemented a sliding window approach with a window size n and a unit step size. For a sequence of length L , this process extracts $L - n + 1$ overlapping sub-sequences, capturing the transitions between system calls that characterize benign or malicious behavior. During the training phase, these chunks are used to define the self space, during the

evaluation phase, they serve as the input for detector matching. To derive a single anomaly metric for an entire trace, the individual scores of these chunks are aggregated via their arithmetic mean.

For a given test sequence S of length L , the sliding window generates $L - n + 1$ overlapping chunks. Each chunk is evaluated against the detector set using the r -contiguous bits matching rule, resulting in an individual anomaly score x_i . To consolidate these results into a single representative value for the entire trace, we calculate the Mean Anomaly Signal, $A(S)$, defined as:

$$A(S) = \frac{1}{L - n + 1} \sum_{i=1}^{L-n+1} x_i$$

This normalizes the impact of sequence length, ensuring that longer traces do not produce disproportionately high scores due to their volume. The resulting value $A(S)$ provides a measure of anomaly density, where a score closer to 1.0 indicates that the majority of the sequence transitions are unrecognizable to the self space.

Negative Selection Algorithm

The negative selection algorithm is inspired by the biological idea of distinguishing *self* from *non-self*. In our setting, the *self space* is defined by fixed-length chunks extracted from normal training traces, and anomalies are sequences that contain many chunks that are “unfamiliar” with respect to this self space.

Concretely, training consists of constructing a repertoire of detectors (also called patterns) of length n over the dataset alphabet (given by the provided .alpha file). A candidate detector is added to the repertoire only if it does *not* match any self chunk under the r -contiguous matching rule: a detector matches a string if the two share at least one contiguous substring of length r at the same position. Detectors that match self are removed, and the remaining repertoire represents patterns that are explicitly *outside* the observed normal behaviour. This corresponds to the standard negative selection procedure described in the assignment.

During evaluation, each window chunk from a test trace is compared against the detector repertoire. We use the count-based scoring, meaning that the anomaly signal for a chunk is the number of detectors that match it. Higher counts indicate that the chunk lies deeper in non-self space and is therefore more anomalous. Since raw counts can be very large, we use the log transform, i.e., $\log_2(1 + x)$, to stabilize the scale. Finally, following our sliding window setup, chunk-level scores are aggregated by their arithmetic mean to obtain a single anomaly score per trace, which is then used for ROC/AUC evaluation.

3 Results

The negative selection algorithm was evaluated using ROC curves and the corresponding AUC values across different parameter settings. The parameter search over several values of n and $r < n$ revealed a clear dependency of detection performance on the balance between detector length and matching threshold (Figure 2).

Shorter detectors tended to produce higher sensitivity but also a higher false positive rate, suggesting that they match many patterns that are not necessarily indicative of anomalous behaviour. In contrast, very long detectors were more specific but occasionally less robust, likely due to overfitting to the training data and reduced generalization to unseen traces. Overall, intermediate detector lengths provided the most stable behaviour.

With respect to the matching parameter r , very small values resulted in overly strict matching, limiting generalization beyond exact substrings observed during training. Conversely, very large values made matching too permissive, causing substantial overlap between normal and anomalous score distributions. The best performance was consistently observed for moderate combinations of n and r , where the model balances discrimination and generalization.

The setting (10, 5) provides a balanced midpoint: detectors are long enough to encode meaningful syscall subsequences, while the matching threshold allows moderate variation within normal behaviour. The combination (11, 6) increases tolerance slightly, enabling detectors to generalize better across natural variability in normal process behaviour, such as minor fluctuations in call ordering or

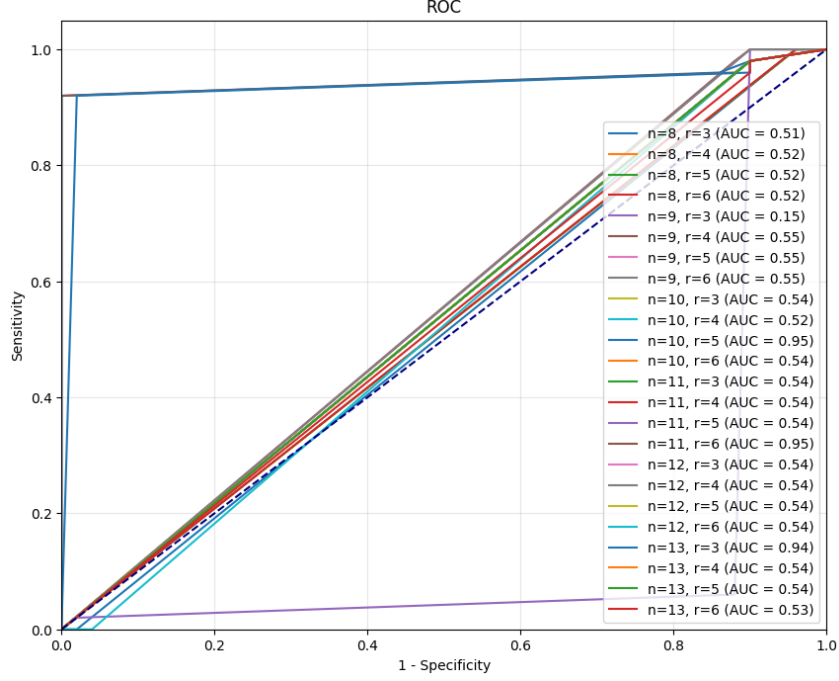


Figure 2: AUC values for distinct (n, r) pairs

frequency. Conversely, $(13, 3)$ uses longer detectors with a stricter matching rule, thereby enforcing stronger structural similarity while still allowing limited deviations.

Importantly, all three parameter choices operate in a regime where detectors capture characteristic local syscall structure but do not overfit to exact training traces. This suggests that successful intrusion detection in this dataset depends on modelling mid-range sequential structure rather than extremely short or extremely long dependencies.

When evaluated on additional datasets (see Figure 3, 4), the selected high-performing configurations maintained good discrimination, although AUC values varied depending on the statistical characteristics of the traces. This indicates that the learned detector repertoire captures structural properties of normal behaviour, yet remains influenced by dataset-specific patterns.

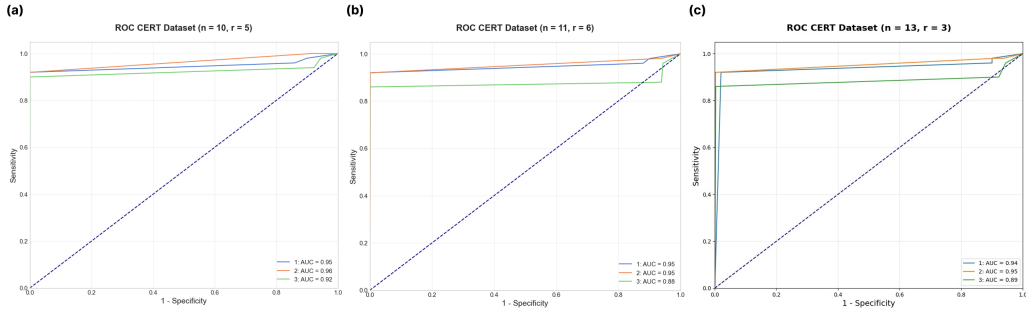


Figure 3: ROC curves for the CERT dataset (Appendix C, Table 3) evaluated at three high-performing parameter sets: (a) $n = 10, r = 5$, (b) $n = 11, r = 6$, and (c) $n = 13, r = 3$. Each panel displays the results for three distinct test traces (1, 2, and 3), demonstrating high discriminative power ($AUC \geq 0.88$).

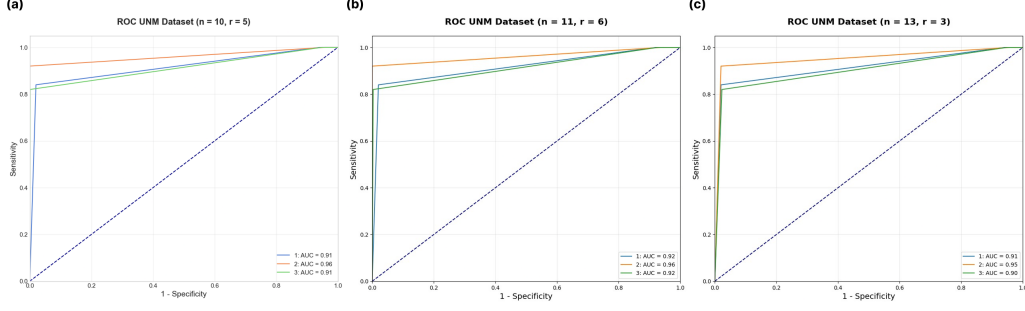


Figure 4: ROC curves for the UNM dataset (Appendix C, Table 4) evaluated at three high-performing parameter sets: (a) $n = 10, r = 5$, (b) $n = 11, r = 6$, and (c) $n = 13, r = 3$. Each panel displays the results for three distinct test traces (1, 2, and 3), demonstrating high discriminative power ($\text{AUC} \geq 0.90$).

Overall, the results confirm that the negative selection approach can detect structural deviations in system call sequences effectively, provided that parameters are chosen within an appropriate range rather than at extreme values.

Finally, regarding the question of what the biggest challenges would be if we had to implement the algorithm ourselves, the main difficulty is efficiency. The space of possible length- n detectors grows exponentially with the alphabet size, so generating candidate detectors and rejecting those that match self can become extremely expensive unless detector generation, matching, and indexing are implemented carefully. In addition, fast r -contiguous matching requires non-trivial data structures (e.g., hashing/indexing of r -grams by position) to avoid comparing each candidate against every self chunk.

4 Discussion and Conclusion

The results demonstrate that the negative selection algorithm is capable of distinguishing normal from anomalous system call sequences when appropriate parameters are chosen. Nevertheless, several methodological limitations and potential improvements should be considered.

A first limitation arises from the sliding window approach. By decomposing variable-length traces into fixed-length substrings of size n , the method focuses exclusively on local structure. This has several consequences. First, windows near the beginning and end of a sequence are underrepresented compared to those in the middle, since central positions appear in more overlapping windows. Second, sequences whose length is not a multiple of n are effectively fragmented, and information at the boundaries between windows is not explicitly modelled. Third, sequences shorter than n are excluded entirely (167 datapoints in our case), which may introduce bias if short traces contain meaningful behavioural patterns.

More fundamentally, the sliding window abstraction removes long-range temporal dependencies. The final anomaly score is obtained by aggregating over window-level scores, which discards ordering information beyond length n . As a result, two traces containing the same local subsequences but arranged differently will receive similar anomaly scores, even if one ordering is abnormal. In this sense, the model captures local deviations but not global behavioural structure.

The parameter analysis further suggests that good performance is not confined to a single sharply defined optimum. The strong results obtained for $(10, 5)$, $(11, 6)$, and $(13, 3)$ indicate the presence of a stable region in parameter space where detector length and matching threshold are well balanced.

In the intrusion detection setting, this balance is particularly important. Normal Unix processes exhibit variability in their system call sequences due to differences in workload, timing, and environment. A matching rule that is too strict would incorrectly flag such benign variability as anomalous, whereas a rule that is too permissive would fail to detect genuine exploits. The fact that multiple configurations achieve high AUC values suggests that the anomaly signal in the data is sufficiently strong at the local subsequence level, and that the algorithm is robust to moderate changes in granularity and tolerance.

Rather than identifying a single optimal pair (n, r) , the results indicate that intrusion detection performance is governed by a trade-off surface. As long as detectors are long enough to encode meaningful syscall patterns and the matching threshold is chosen proportionally, the negative selection algorithm can reliably distinguish normal from anomalous behaviour.

In conclusion, the negative selection algorithm provides a conceptually simple and biologically inspired approach to anomaly detection in sequential data. It performs well when local deviations are informative and when the training data sufficiently captures normal variability. However, its reliance on fixed-length windows and its sensitivity to parameter choice and dataset properties limit its ability to model complex temporal structure. Future improvements could combine negative selection with richer sequence models or more principled dataset construction strategies to enhance robustness and generalization.

References

- [1] S. A. Hofmeyr, S. Forrest, and A. Somayaji. Intrusion detection using sequences of system calls. *J. Comput. Secur.*, 6(3):151–180, July 1998.
- [2] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

A Language Classification Toy Example

Setup and ROC/AUC computation

We trained the repertoire on `english.train` with substring length $n = 10$. For scoring, we used the count-based score $(-c)$ and the log transform $\log_2(1 + x)$ (-1) . Each test string (one line) yields one anomaly score. ROC curves were computed by merging English test strings (normal) and the other-language strings (anomalous), sorting by anomaly score, and sweeping all distinct cut-offs; at each cut-off we compute sensitivity (TPR) and $1 - \text{specificity}$ (FPR). We were careful with tied scores and included the endpoints $(0,0)$ and $(1,1)$.

Q1: Which r belongs to the AUCs in Figure 1?

The first figure in the assignment matches $r = 1$. The second figure with $\text{AUC} = 0.59$ is the one with $r = 7$. The last figure with $\text{AUC} = 0.79$ is the one with $r = 4$.

Q2: How does AUC change with r (English vs Tagalog)?

Initially, increasing r (the number of mismatches/switches allowed when matching length-10 substrings against the English training data) improves the AUC. Going from $r = 1$ to $r = 3$, the AUC rises from about 0.54 (only slightly better than random guessing, $\text{AUC} = 0.5$) to about 0.83, meaning the classifier separates English from Tagalog much better. This likely means that when we allow some bigger number of switches we are more likely to generalize on English better as then our train data doesn't have to have all of the possible strings of length 10, but we use up the general pattern of English to separate it from the Tagalog texts.

However, when r becomes too large, the AUC starts to drop again. With high values (e.g., $r = 8-9$), the matching becomes so permissive that almost any substring can be considered close enough to something in English. This makes the score distributions for English and Tagalog overlap heavily, so the classifier loses separation power. At $r = 1$ we see low performance as we are too strict only allowing small perturbances from the train sample. With $r = 9$ we see lowest performance ($\text{AUC} \approx 0.51$) as we allow almost all of the changes in the string making almost all of the Tagalog words look like English. Overall, the best discrimination is achieved at a moderate $r = 3$.

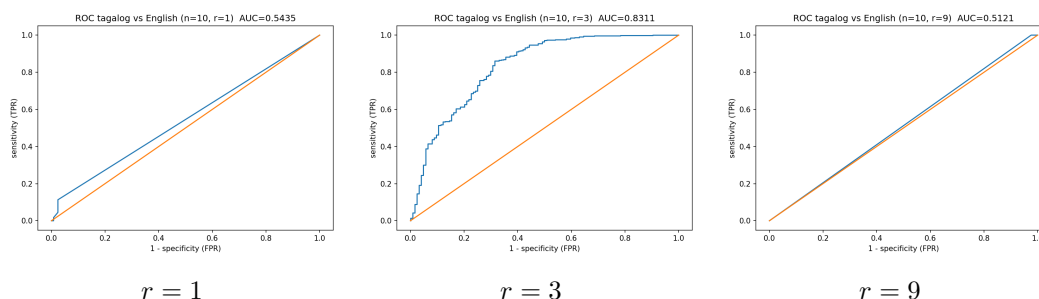


Figure 5: Tagalog vs English ROC curves showing the strict ($r = 1$), best ($r = 3$), and overly permissive ($r = 9$) regimes.

Q3: Which languages are easiest/hardest vs English?

Across all languages, the best AUC was always achieved for $r = 3$, suggesting this value gives the best balance between being strict enough to capture “English-ness” and flexible enough to generalize beyond exact substrings. The negative selection model separates Xhosa best from English (best $\text{AUC} = 0.89$), followed by Hiligaynon (best $\text{AUC} = 0.84$) and Tagalog (best $\text{AUC} = 0.83$), with Plautdietsch still reasonably separable (best $\text{AUC} = 0.775$). The most difficult case is Middle English (best $\text{AUC} = 0.54$), which is close to random performance, likely because its character patterns and short substrings overlap heavily with modern English, causing strong score overlap and weak ROC separation.

Language (vs English)	Best AUC	Best r
Xhosa	0.89	3
Hiligaynon	0.84	3
Tagalog	0.83	3
Plautdietsch	0.78	3
Middle English	0.54	3

Table 1: Best observed discrimination (AUC) per language.

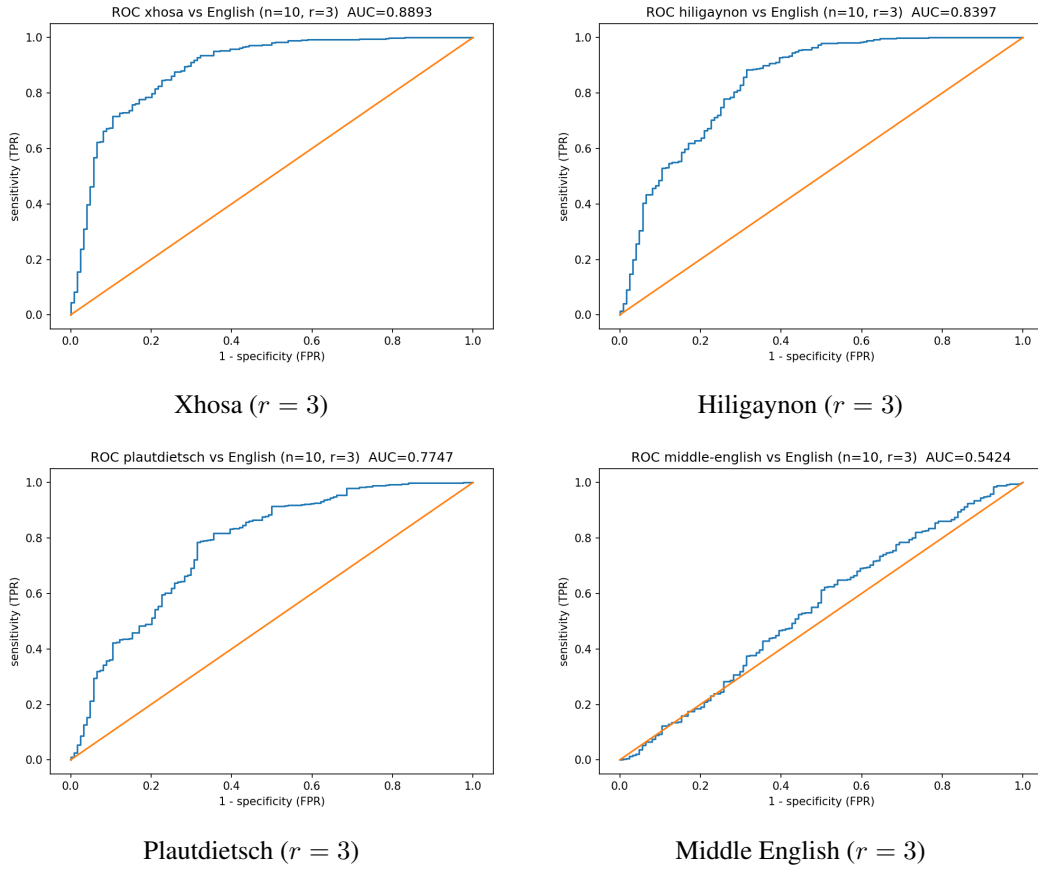


Figure 6: ROC curves at $r = 3$ for the additional languages in lang/.

Q4: Why too short/too long inputs are problematic

The method is based on checking length- n substrings, so if the input is shorter than n there are no such substrings to match and the score becomes trivial and uninformative. If the input is much longer, it contains many more length- n substrings, which creates many more chances for matches and makes the score grow mainly with length rather than with how English-like the content is, so scores across examples stop being comparable unless you normalize for length.

B Parameters

Table 2: Negative Selection Algorithm Performance across Parameter Grid

Experiment	Window Size (n)	Match Threshold (r)	AUC Score
Set 1	8	3, 4, 5, 6	0.51 – 0.52
Set 2	9	3, 4, 5, 6	0.15 – 0.55
Set 3	10	5	0.95
Set 4	10	3, 4, 6	0.52 – 0.54
Set 5	11	3, 4, 5	0.54
Set 6	11	6	0.95
Set 7	12	3, 4, 5, 6	0.54
Set 8	13	3	0.94
Set 9	13	4, 5, 6	0.53 – 0.54

C Performance on Datasets

Table 3: Performance Comparison Across Top Parameter Sets and CERT Sub-datasets

Window Size (n)	Match Threshold (r)	Dataset 1 (AUC)	Dataset 2 (AUC)	Dataset 3 (AUC)
10	5	0.95	0.96	0.92
11	6	0.95	0.95	0.88
13	3	0.94	0.95	0.89

Table 4: Performance Comparison Across Top Parameter Sets and UNM Sub-datasets

Window Size (n)	Match Threshold (r)	Dataset 1 (AUC)	Dataset 2 (AUC)	Dataset 3 (AUC)
10	5	0.91	0.96	0.91
11	6	0.92	0.96	0.92
13	3	0.91	0.95	0.90